

Типы данных в API

Справочник тестировщика: типы JSON и OpenAPI, их форматы и что с ними проверять

База: типы данных JSON

Любой ответ API — это JSON, а в JSON всего шесть типов:

Тип	Пример
string	"John"
number	25.5
object	{ "id": 1 }
array	[1, 2, 3]
boolean	true
null	null

OpenAPI уточняет их через ключевые слова: `integer` — целое, `format` — подтип (дата, uuid, email...), а также ограничения: `minimum`, `maxLength`, `pattern`, `enum` и другие. Типы полей описаны в документации — это часть контракта API, и каждое отклонение от них — баг.

String — строка

```
"name": "John Doe"
```

Форматы (format)

- `date` — дата: `"2026-08-23"`
- `date-time` — дата и время: `"2026-08-23T12:00:00Z"`
- `email`, `uuid`, `uri`, `ipv4`, `hostname` — по соответствующим стандартам

Что проверять

- Длина: `minLength` / `maxLength` — пустая строка, 1 символ, граница, превышение
- `pattern` — регулярка допустимых символов; цифры, спецсимволы, другая раскладка
- Пробелы по краям (тримминг), регистр, кодировки и юникод

Integer — целое число

```
"age": 25
```

Форматы

- `int32` — от -2 147 483 648 до 2 147 483 647
- `int64` — от -9 223 372 036 854 775 808 до 9 223 372 036 854 775 807

Что проверять

- Границы: `minimum` / `maximum` и значения сразу за ними
- Ноль, отрицательные, очень большие числа (переполнение `int32`)
- Дробное значение 25.5 — не целое; число строк `"25"` — классический баг типа в ответе

Number — дробное число

```
"price": 19.99
```

Форматы

- `float` — одинарная точность (~7 значащих цифр)
- `double` — двойная точность (~15 значащих цифр)

Что проверять

- Точка как разделитель, количество знаков после запятой, округление
- Отрицательные, ноль, экспоненциальная запись
- `multipleOf` — кратность (например, шаг 0.01 для цен)

Boolean — логическое

```
"is_active": true
```

- Только `true` или `false`, без кавычек: `"true"` строкой — баг
- Не путать с числами: 0/1 вместо `false/true` — частое расхождение
- Значение по умолчанию, если поле не прислано

Array — массив

```
"items": [ {"id": 1}, {"id": 2} ]
```

- Элементы одного типа по схеме (`items`)
- Пустой массив `[]` — валидный ответ, когда данных нет; не ошибка
- Ограничения `minItems` / `maxItems` , порядок, дубликаты
- Пагинация и сортировка для списков — отдельная зона проверок

Object — объект

```
"user": { "id": 1, "name": "John", "email": null }
```

- Обязательные ключи (`required`) — отсутствие обязательного поля это баг
- Лишние поля (`additionalProperties`) — пропускает сервер или режет?
- Вложенные объекты — типы проверяются на каждом уровне
- Ключи регистрозависимы: `"Name"` и `"name"` — разные поля

Null — отсутствие значения

```
"middle_name": null
```

- `null ≠ 0`, `null ≠ ""`, `null ≠ []` — не подменяй при проверке
- Где `null` допустим — решает документация: `nullable: true` или `type: [string, null]`
- Отличай «поле отсутствует» от «поле = null» — сервисы часто трактуют по-разному

Enum — перечисление

```
"status": "active" // допустимы только: active, blocked, deleted
```

- Только значения из списка; всё остальное — ошибка 400
- Регистр: "ACTIVE" не должен проходить, если задокументирован нижний
- Проверь каждое значение списка — все должны обрабатываться

Дата и время

```
"created_at": "2026-08-23T12:00:00Z"
```

- Стандарт ISO 8601 / RFC 3339: YYYY-MM-DDTHH:MM:SSZ
- Таймзоны: суффикс Z (UTC) или смещение +03:00
- Границы: даты в прошлом и будущем, високосные годы, точность (с секундами, миллисекундами или без)
- Сроки действия — любимое место багов: «истёкшее» значение всё ещё активно

UUID и другие идентификаторы

```
"id": "550e8400-e29b-41d4-a716-446655440000"
```

- UUID (RFC 4122): 36 символов, 8-4-4-4-12, шестнадцатеричные
- Формат кодов часто задан жёстко (префикс, регистр, длина) — сверяй с документацией
- Целочисленные ID: отрицательные, нулевые, очень большие, несуществующие

Комбинации схем

- `oneOf` — ровно одна из схем
- `anyOf` — хотя бы одна из схем
- `allOf` — все схемы одновременно

Встречаются в сложных ответах: поле может быть строкой ИЛИ объектом. Проверь все ветви каждой комбинации.

Типы данных — часть контракта API

Документация описывает не только методы, но и типы каждого поля. Любое отклонение — баг.

Больше полезного про тестирование — в телеграм-канале [@eddytester](#).